

TP: Méthodes de gradient

1. Ecrire une fonction python qui calcule cette fonction:

```
def rosenbrock(x,y):  
    return (x-1)**2+100*(x**2-y)**2
```

2. Calculer le gradient de cette fonction et donner une fonction python qui calcule ce gradient:

$$\frac{\partial f}{\partial x} = 2(x-1) + 400x(x^2 - y)$$

$$\frac{\partial f}{\partial y} = -200(x^2 - y)$$

$$\nabla f(x,y) = \begin{pmatrix} 2(x-1) + 400x(x^2 - y) \\ -200(x^2 - y) \end{pmatrix}$$

```
def gradient_rosenbrock(x, y):  
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)  
    dfdy = -200*(x**2 - y)  
    return np.array([dfdx, dfdy])
```

3. Tracer les lignes de niveaux de f dans le rectangle $(x, y) \in [-1.5, 1.5] \times [-0.5, 1.5]$. On utilisera la fonction `plt.contour`

```
import matplotlib.pyplot as plt  
import numpy as np  
x = np.linspace(-1.5, 1.5, 400)  
y = np.linspace(-0.5, 1.5, 400)  
  
X, Y = np.meshgrid(x, y)  
Z = (X-1)**2 + 100*(X**2 - Y)**2  
niveaux= [0.05,0.2, 0.5, 1, 2, 4, 8, 16, 32, 64, 128, 256, 512]  
plt.contour(X, Y, Z, levels=niveaux)  
plt.show()
```

4. Méthode de gradient à pas fixe et critères d'arrêt .

```
import numpy as np
def gradient_rosenbrock(x, y):
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)
    dfdy = -200*(x**2 - y)
    return np.array([dfdx, dfdy])

def gradient_pas_fixe(x, y, h, tol=1e-3, itmax=100000):
    for k in range(itmax):
        g = gradient_rosenbrock(x, y)
        if np.sqrt(g[0]**2 + g[1]**2) <= tol:
            break
        x = x - h*g[0]
        y = y - h*g[1]
    return x, y, k
```

Critère 1 (précision) : on s'arrête si la longueur du gradient est inférieure à 0,001, soit $\|\nabla f(x_k)\| \leq 0,001$.

Critère 2 (sécurité) : on s'arrête de toute façon après itmax = 100 000 itérations, même si le critère 1 n'est jamais atteint. Sens : garantir que le programme se termine toujours, même si h est mal choisi et que la méthode oscille ou avance trop lentement.

5.

```
import numpy as np
import matplotlib.pyplot as plt
def gradient_rosenbrock(x, y):
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)
    dfdy = -200*(x**2 - y)
    return np.array([dfdx, dfdy])

def gradient_pas_fixe(x, y, h, tol=1e-3, itmax=100000):
    xs = [x]
    ys = [y]
    for k in range(itmax):
        g = gradient_rosenbrock(x, y)
        if np.sqrt(g[0]**2 + g[1]**2) <= tol:
            break
```

```

        x = x - h*g[0]
        y = y - h*g[1]
        xs.append(x)
        ys.append(y)
        plt.plot(xs, ys)
        plt.show()
        return x, y, k
x, y, k = gradient_pas_fixe(-1, 1, h=1e-3)
for h in [1e-4, 1e-3, 1.9e-3, 2.5e-3]:
    gradient_pas_fixe(-1, 1, h)

```

Pour $h = 10^{-4}$, 10^{-3} et $1,9 \cdot 10^{-3}$, la méthode converge vers le minimum (1, 1) : la trajectoire descend d'abord au fond de la vallée, puis suit la parabole $y = x^2$ jusqu'au minimum. Le nombre d'itérations dépend fortement de h : 147 512 itérations pour $h = 10^{-4}$, 14 748 pour $h = 10^{-3}$ et 7 761 pour $h = 1,9 \cdot 10^{-3}$. Plus h est petit, plus la méthode est lente.

Pour $h = 2,5 \cdot 10^{-3}$, la méthode ne converge plus : les pas sont trop grands, le point saute d'un versant de la vallée à l'autre et oscille sans jamais atteindre le minimum (arrêt par le critère de sécurité itmax).

Conclusion : le pas h est un compromis. Trop petit, la convergence est très lente ; trop grand (au-delà d'un seuil situé vers $h \approx 2 \cdot 10^{-3}$), la méthode devient instable et diverge. Il faut donc choisir h assez grand pour aller vite, mais en dessous du seuil de stabilité.

6.

```

import numpy as np
import matplotlib.pyplot as plt
def rosenbrock(x,y):
    return (x-1)**2 + 100*(x**2 - y)**2
def gradient_rosenbrock(x, y):
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)
    dfdy = -200*(x**2 - y)
    return np.array([dfdx, dfdy])
def gradient_pas_1k(x, y, itmax=20):
    xs = [x]; ys = [y]
    for k in range(1, itmax + 1):
        g = gradient_rosenbrock(x, y)
        if not np.all(np.isfinite(g)):
            # k = 1, 2, 3, ...
            # inf ou nan : ça a

```

```

explose, on stoppe
    break
    alpha = 1/k                                # le pas de ce tour
    x = x - alpha*g[0]
    y = y - alpha*g[1]
    xs.append(x); ys.append(y)
    print(f"k={k}  alpha={alpha:.3f}  x=({x:.4g}, {y:.4g})
f={rosenbrock(x, y):.4g}")
    return x, y, k

gradient_pas_1k(-1, 1)

```

Avec le pas prédéterminé $\alpha_k = 1/k$, la méthode diverge : les valeurs explosent et dépassent la capacité de l'ordinateur en 5 itérations seulement. La cause est le tout premier pas, $\alpha_1 = 1$, beaucoup trop grand pour cette fonction : il envoie le point en (3, 1), très loin dans une zone où le gradient de Rosenbrock est énorme (il croît comme $400 \cdot x^3$). À partir de là, le gradient grossit à chaque itération bien plus vite que le facteur $1/k$ ne diminue, et chaque pas amplifie le précédent au lieu de le corriger. Conclusion : un pas décroissant n'est pas une garantie de convergence — s'il démarre trop grand par rapport à la raideur de la fonction, la méthode explose. Sur ce problème, le pas fixe bien réglé de la question 5 est nettement meilleur.

```

k=1  alpha=1.000  x=(3, 1)  f=6404
k=2  alpha=0.500  x=(-4799, 801)  f=5.304e+16
k=3  alpha=0.333  x=(1.474e+13, 1.535e+09)  f=4.715e+54
k=4  alpha=0.250  x=(-3.2e+41, 1.086e+28)  f=1.048e+168
k=5  alpha=0.200  x=(2.621e+126, 4.096e+84)  f=inf
C:\Users\Mohamed\AppData\Local\Temp\ipykernel_41644\2887415235.py:4: RuntimeWarning: overflow encountered in scalar power
    return (x-1)**2 + 100*(x**2 - y)**2
C:\Users\Mohamed\AppData\Local\Temp\ipykernel_41644\2887415235.py:6: RuntimeWarning: overflow encountered in scalar multiply
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)
(np.float64(2.6210355926919487e+126), np.float64(4.095578731555529e+84), 6)

```

7.

```

import numpy as np
import matplotlib.pyplot as plt
def rosenbrock(x,y):
    return (x-1)**2 + 100*(x**2 - y)**2
def gradient_rosenbrock(x, y):
    dfdx = 2*(x - 1) + 400*x*(x**2 - y)

```

```

    dfdy = -200*(x**2 - y)
    return np.array([dfdxd, dfdy])
def gradient_normalise(x, y, h=0.05, nit=200):
    xs = [x]; ys = [y]
    for k in range(nit):
        # exactement 200
        iterations
        g = gradient_rosenbrock(x, y)
        longueur = np.sqrt(g[0]**2 + g[1]**2)
        x = x - h*g[0]/longueur
        # pas de longueur h
        exactement
        y = y - h*g[1]/longueur
        xs.append(x); ys.append(y)

    return x, y, k

x, y, k = gradient_normalise(-1, 1)
print(x, y, k)

```

après 200 itérations, on arrive en (0,38 ; 0,11) avec $f \approx 0,50$ — encore loin du minimum (1, 1).

8.

On constate que la méthode est rapide au début (pas de longueur constante $h = 0,05$, elle avance vite même quand la pente est faible), mais elle ne converge pas : près du minimum, le pas devient trop grand par rapport à la largeur de la vallée, le point oscille autour du minimum sans l'atteindre et f stagne vers 0,25. Sa précision est donc limitée par la taille du pas h ; pour converger vraiment, il faudrait diminuer h au fil des itérations.